

DSA CODING ASSIGNMENT 1

Name – Sunny Singh

Section – FB

Uni Roll No. – 12515001642

Class Roll No. - 64

Q. 1464. Maximum Product of Two Elements in an Array

```
class Solution {  
    public int maxProduct(int[] nums) {  
        int first =0;  
        int second =0;  
        for(int n:nums)  
        {  
            if(n>=first)  
            {  
                second =first;  
                first=n;  
  
            }  
            else if(n>second)  
                second=n;  
  
        }  
        return (first-1)*(second-1);  
    }  
}
```



Q. Second-Largest

```
class Solution {
```

```
    public int getSecondLargest(int[] arr) {
```

```
        int largest = -1;
```

```
        int second = -1;
```

```
        for (int i = 0; i < arr.length; i++) {
```

```
            if (arr[i] > largest) {
```

```
                second = largest;
```

```
                largest = arr[i];
```

```
            }
```

```

        else if (arr[i] > second && arr[i] != largest) {
            second = arr[i];
        }
    }

    return second;
}
}

```

The screenshot shows a coding platform interface. On the left, the 'Output Window' is open, displaying 'Compilation Results' for a custom input 'Y.O.G.I. (AI Bot)'. It indicates the problem was solved successfully with 1120/1120 test cases passed, 1/1 attempts correct, 100% accuracy, 2/2 points scored, and a time taken of 0.97. On the right, the Java code for the 'getSecondLargest' method is shown, which implements the logic for finding the second largest element in an array.

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
class Solution {
    public int getSecondLargest(int[] arr) {
        int largest = -1;
        int second = -1;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > largest) {
                second = largest;
                largest = arr[i];
            }
            else if (arr[i] > second && arr[i] != largest) {
                second = arr[i];
            }
        }

        return second;
    }
}

```

Q. 162. Find Peak Element

```

class Solution {
    public int findPeakElement(int[] nums) {
        int left = 0;
        int right = nums.length - 1;
        while (left < right) {

```

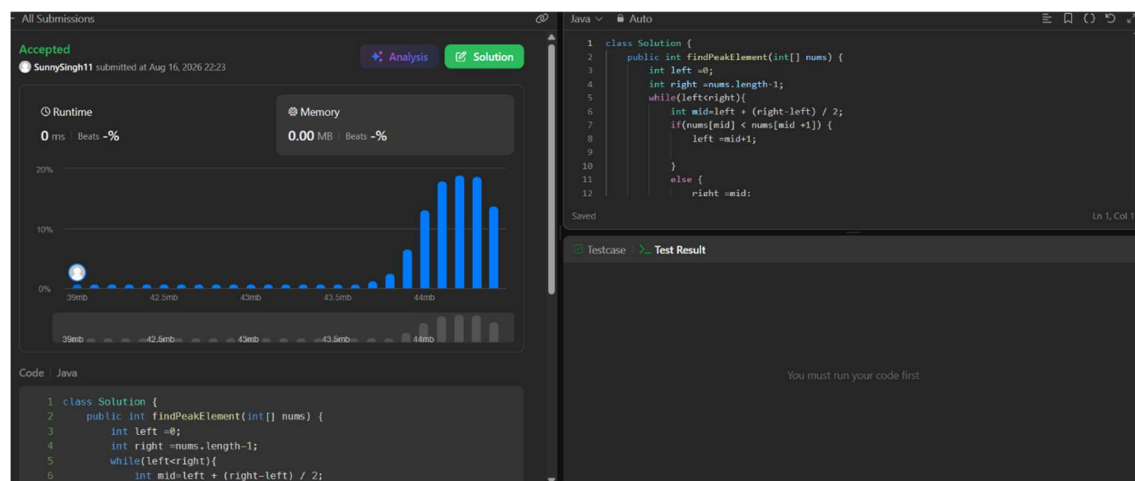
```

        int mid=left + (right-left) / 2;
        if(nums[mid] < nums[mid +1]) {
            left =mid+1;

        }
        else {
            right =mid;
        }
    }
    return left;

}
}

```



Q. 33. Search in Rotated Sorted Array

```

class Solution {

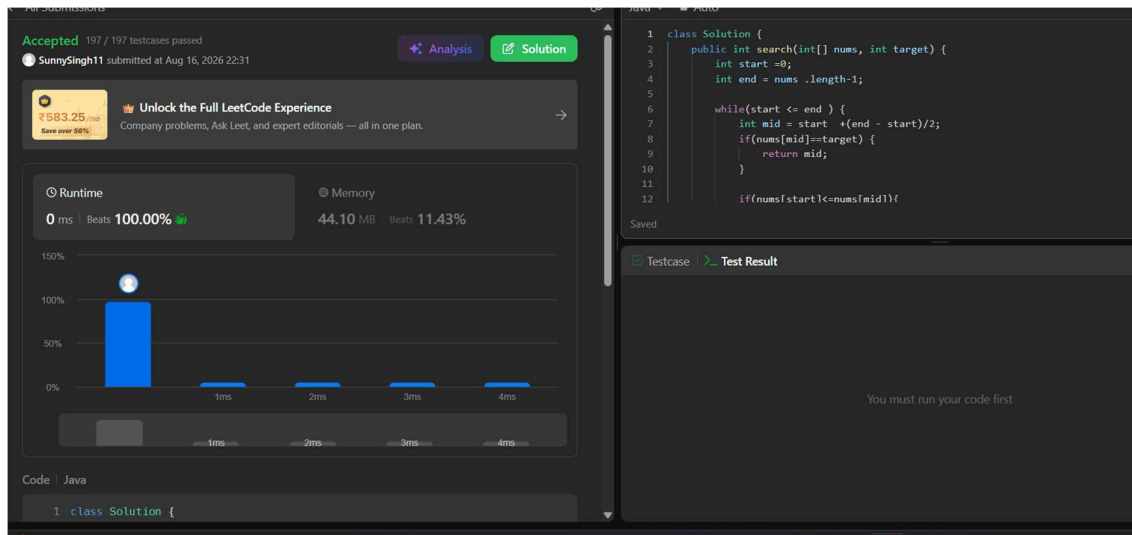
```

```
public int search(int[] nums, int target) {  
    int start =0;  
    int end = nums .length-1;  
  
    while(start <= end ) {  
        int mid = start +(end - start)/2;  
        if(nums[mid]==target) {  
            return mid;  
        }  
  
        if(nums[start]<=nums[mid]){  
            if(nums[start] <= target && target < nums[mid]) {  
                end = mid -1;  
  
            } else {  
                start =mid +1;  
  
            }  
        } else {  
            if(nums[mid]<target && target <= nums[end]) {  
                start = mid+1;  
            }  
        }  
    }  
}
```

```

    } else {
        end = mid-1;
    }
}
}
return -1 ;
}
}

```



Q.3477. Fruits Into Baskets II

```

class Solution {
    public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
        int fru_size=fruits.length;

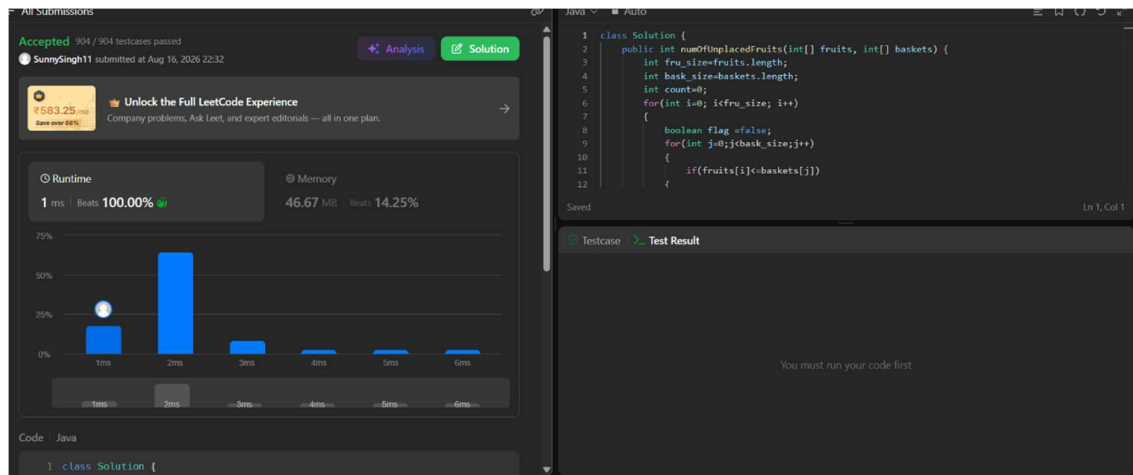
```

```
int bask_size=baskets.length;
int count=0;
for(int i=0; i<fru_size; i++)
{
    boolean flag =false;
    for(int j=0;j<bask_size;j++)
    {
        if(fruits[i]<=baskets[j])
        {
            baskets[j]=-1;
            flag=true;
            break;
        }
    }
    if(!flag)
    {
        count+=1;
    }
}

return count ;
```


}

}



Q. Best Time to Buy and Sell Stock

class Solution {

public int maxProfit(int[] prices) {

int minPrice = prices[0];

int profit = 0;

for (int i = 1; i < prices.length; i++) {

if (prices[i] < minPrice) {

minPrice = prices[i];

} else {

profit = Math.max(profit, prices[i] - minPrice);

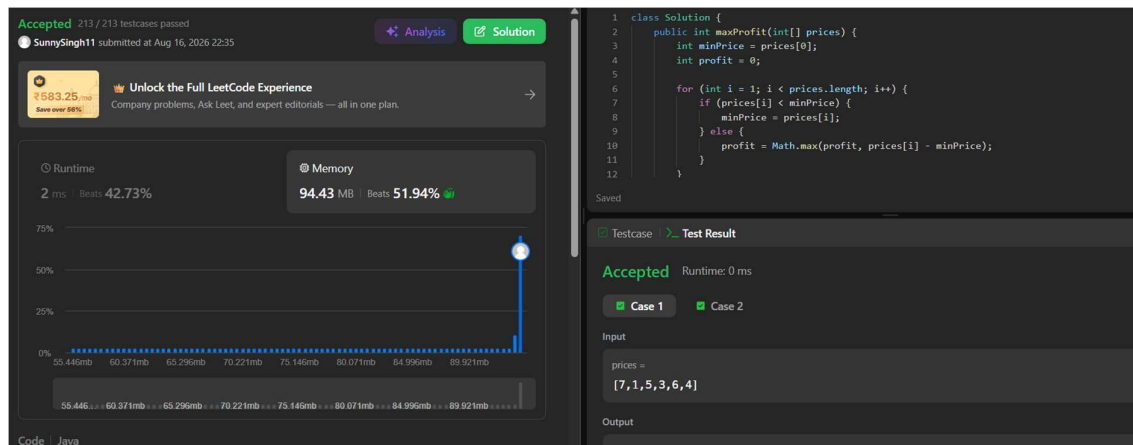
}

}

```

        return profit;
    }
}

```



Q. 1. Two Sum

```

class Solution {

```

```

    public int[] twoSum(int[] nums, int target) {

```

```

        for (int i = 0; i < nums.length; i++) {

```

```

            for (int j = i + 1; j < nums.length; j++) {

```

```

                if (nums[i] + nums[j] == target) {

```

```

                    return new int[] {i, j};
                }
            }
        }
    }
}

```

```
}
```

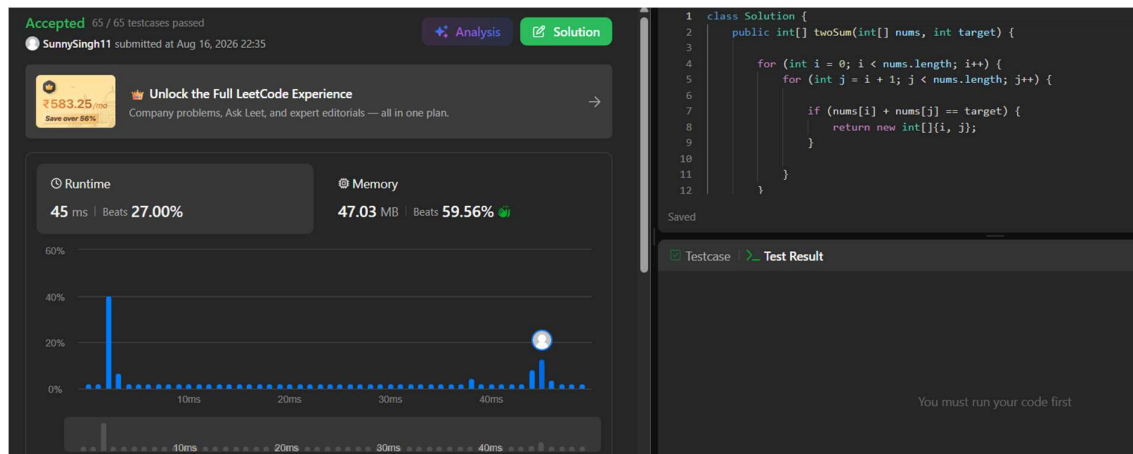
```
}
```

```
}
```

```
return new int[] {};
```

```
}
```

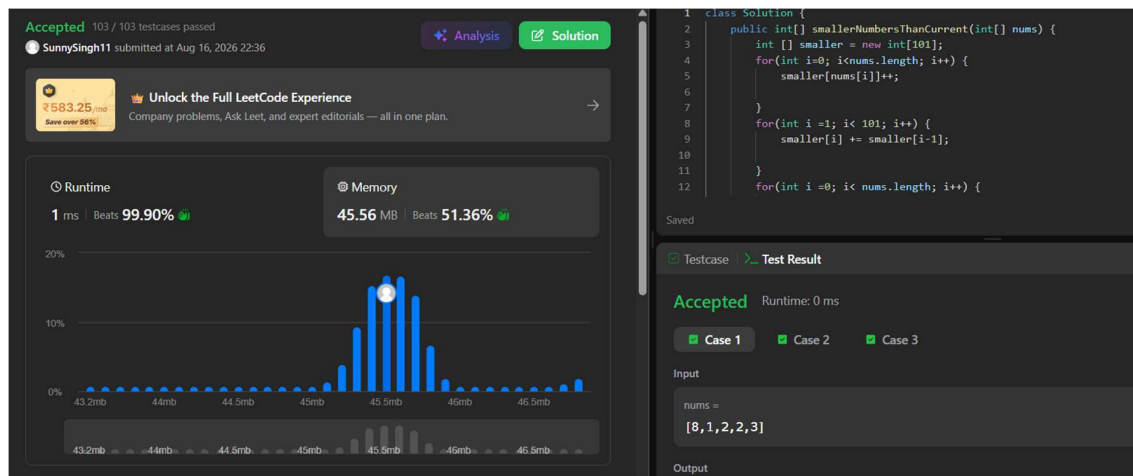
```
}
```



Q. 1365. How Many Numbers Are Smaller Than the Current Number

```
class Solution {  
    public int[] smallerNumbersThanCurrent(int[] nums) {  
        int [] smaller = new int[101];  
        for(int i=0; i<nums.length; i++) {  
            smaller[nums[i]]++;  
        }  
    }  
}
```

```
}  
for(int i =1; i< 101; i++) {  
    smaller[i] += smaller[i-1];  
  
}  
for(int i =0; i< nums.length; i++) {  
    int position=nums[i];  
    if(position==0) nums[i]=0;  
    else {  
        nums[i] = smaller[position-1];  
    }  
}  
return nums;  
  
}  
}
```



Q. 628. Maximum Product of Three Numbers

```
class Solution {
```

```
    public int maximumProduct(int[] nums) {
```

```
        Arrays.sort(nums);
```

```
        int n = nums.length;
```

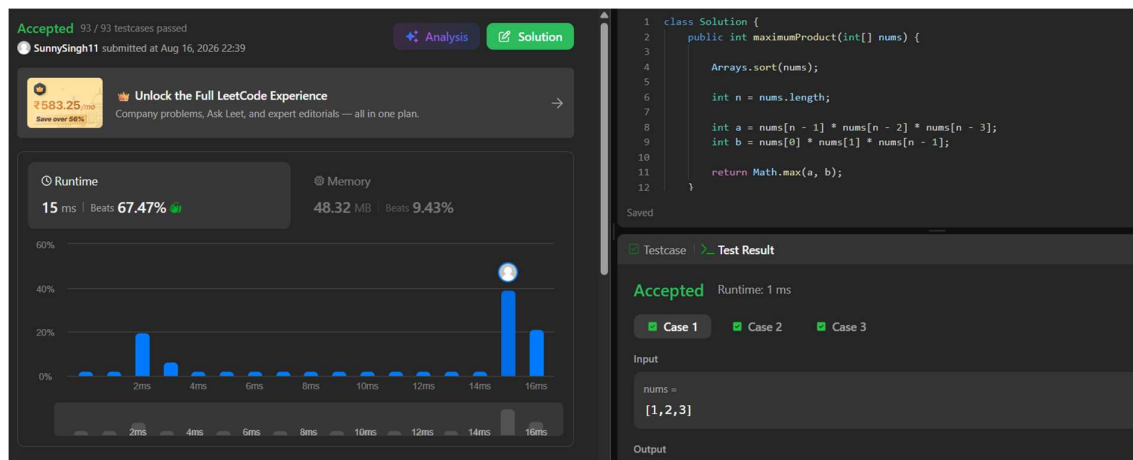
```
        int a = nums[n - 1] * nums[n - 2] * nums[n - 3];
```

```
        int b = nums[0] * nums[1] * nums[n - 1];
```

```
        return Math.max(a, b);
```

```
    }
```

```
}
```



Q.26. Remove Duplicates from Sorted Array

```
class Solution {  
    public int removeDuplicates(int[] nums) {  
        int j = 0;  
  
        for (int i = 1; i < nums.length; i++) {  
            if (nums[i] != nums[j]) {  
                j++;  
                nums[j] = nums[i];  
            }  
        }  
  
        return j + 1;  
    }  
}
```

}

}

